

03 Components with Known Vulnerabilities

This kind of risk occurs when an application makes use of libraries, frameworks, or plugins that have publicly documented flaws which have never been patched. Attackers can find out about the vulnerability and then make use of an existing exploit. In my tests, I determined the component and its version, for instance, by examining the headers or by looking at the scan results, and then checked a database such as CVE or Exploit-DB to locate a match. Having obtained the available exploit, I was able to gain access or execute commands. The findings demonstrated that outdated components are easy targets, so it is important to update them regularly and keep an eye on dependencies.

04 Cross-Site Request Forgery

A CSRF attack causes a browser already logged into a website to submit an unwanted request, resulting in actions being carried out without the user's knowledge. For instance, when I made a malicious link that changed the user's password, it showed that the application lacked anti-CSRF tokens.

05 Directory Traversal

Directory traversal enables an attacker to alter file paths, for instance by using sequences of `../`, in order to access files that are located outside the intended folder. When I altered a URL parameter by adding `../..` I was then able to, for example, read `/etc/passwd`, which shows that the application did not validate user input.

06 DOM XSS

DOM XSS happens when client-side JavaScript takes untrusted input, for example a URL fragment, and improperly includes it on the page, which allows attackers to run scripts in the victim's browser. For instance, when I entered a payload into the search field and then triggered an alert, this demonstrated that the input had not been sanitized.

08 Header Injection

Header injection happens when user input is included in an HTTP header without it being filtered, which enables attackers to insert line breaks in order to add headers or alter the responses. In the laboratory, I inserted a new line into a parameter and, for example, injected a custom header, thus demonstrating that the input had not been validated.

09 Horizontal Privilege Escalation

Horizontal privilege escalation happens when a user is able to access another user's data or functions while having the same level of permissions, usually since the application does not check who owns the data. For instance, in the laboratory I altered the user ID in the URL and then viewed the details of another account.

12 Log4j 2 CVE-2021-44228

Log4Shell is a vulnerability in Log4j 2 that lets an attacker make the server retrieve and execute remote code by using attacker-controlled input that contains a JNDI string for logging. For instance, when I ran some tests, like sending a JNDI payload in a header, I got a callback, which proved that remote code execution was possible.

13 PII Data in URL

This risk arises when an application puts personal information such as names, email addresses, or IDs into URL parameters, exposing it through browser history, server logs, and referrer headers. For example, in the lab, I looked at the address bar after logging in and saw my email address in plain text.

Ruby rest-client 1.6.13 backdoor vulnerability

The malicious code was added to this version of the rest-client Ruby gem after the maintainer's account was compromised, allowing attackers to run remote commands and steal credentials. To show the risks of compromised dependencies, I downloaded the versioned gem and, for versioned, removed the backdoor.

16 Server-Side Request Forgery

An attacker can trick a server into making requests on their behalf, usually to internal systems or cloud metadata services that are normally inaccessible from outside. For example, in the lab, I accessed an internal admin page by changing a URL parameter to `http://localhost`.

17 Session Fixation

Session fixation happens when an application fails to assign a new session ID after a user log in, allowing an attacker to choose a known session ID for the victim and then use it to take over the account. In my tests, for example, I provided a fixed session ID and obtained access after the victim had logged in.

20 TikTok Cross-Site Scripting Vulnerability

This flaw enabled attackers to insert dangerous JavaScript into a TikTok web page, which would then run in the victim's browser and could be used to steal session data or take control of accounts. For instance, when I put a script payload into an input field and for example caused an alert to appear, it showed that there was no sanitization.

24 Weak Randomness

Weak randomness happens when an application uses predictable values to generate tokens, session IDs, or passwords, which allows attackers to guess or reproduce them. For example, in the laboratory I gathered a number of the generated tokens and for example spotted a pattern that could be used to predict the next one, demonstrating that the values were not genuinely random.